



1

Variational Autoencoders

Prof Osama Abdel Raouf

Decisions and Knowledge

Decisions

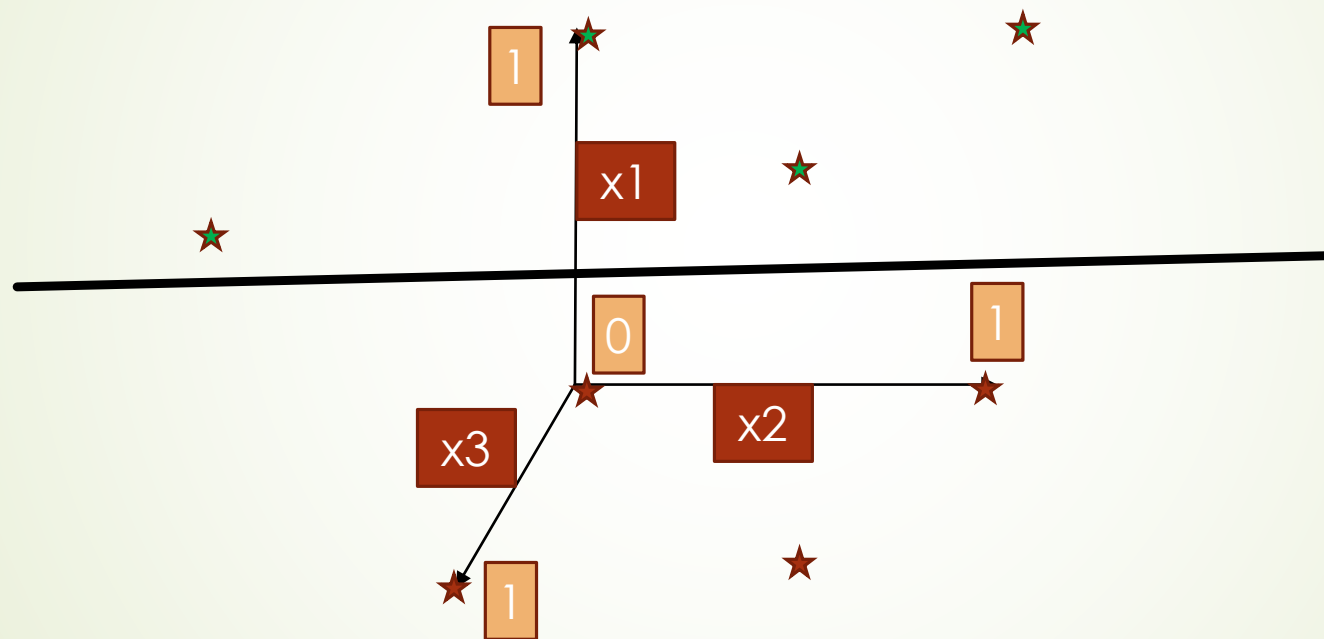
- Say you want to decide whether you are going to attend a festival this upcoming weekend. There are three variables that go into your decision
 1. X1= Is the weather good? **[0=bad,1=good]**
 2. X2= Does your friend want to go with you? **[0=no,1=yes]**
 3. X3= Is it near public transportation? **[0=no,1=yes]**

Knowledge

- Your Decisions Might be like this:

x1	x2	x3	Y
0	0	0	No
0	0	1	No
0	1	0	No
0	1	1	No
1	0	0	Yes
1	0	1	Yes
1	1	0	Yes
1	1	1	Yes

Visual Representation



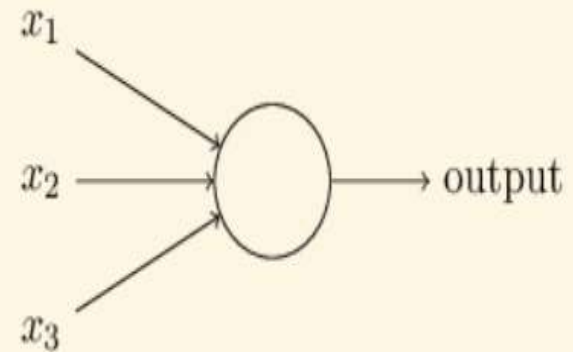
Decision Boundary/
Hyperplane

How to Simplify It for Neural Networks

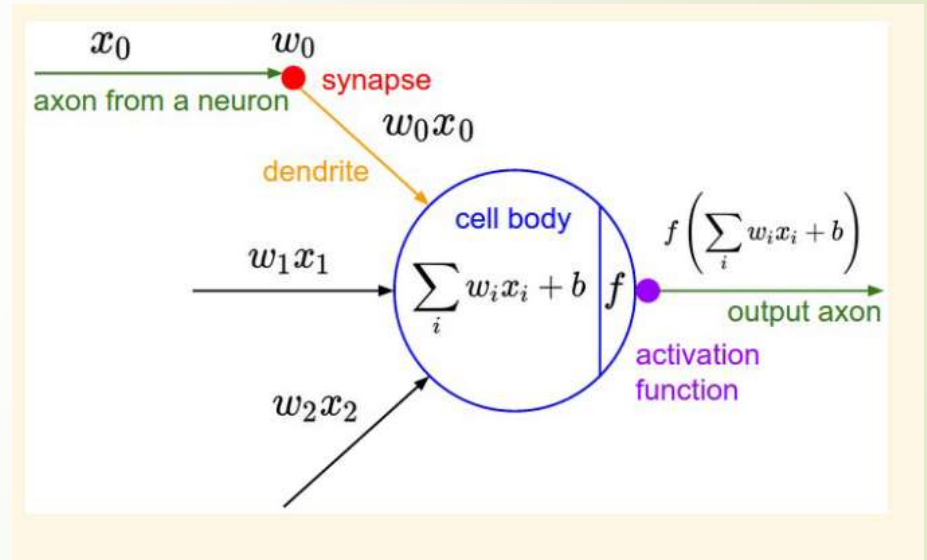
- Using Equations:
 - $Y = 0, (x_1 * w_1 + x_2 * w_2 + x_3 * w_3) + b < \text{threshold}$
 - $Y = 1, (x_1 * w_1 + x_2 * w_2 + x_3 * w_3) + b \geq \text{threshold}$
- If we replace individual variables using vectors, s.t.
 - $X = [x_1, x_2, x_3], W = [w_1, w_2, w_3], B = [b_1, b_2, b_3]$ [B is bias, that shift the plane towards origin]
 - $Y = 0, XW + B < 0$
 - $Y = 1, XW + B \geq 0$
 - The same hyperplane is applied to logistic regression

What is a Neuron

Perceptron



How it works



How the Activation Function Works (An Example with Sigmoid Function)

- ▶ Normal activation function will just through 0 or 1. If value does not cross a threshold it will through 0 else 1.
- ▶ When we use Sigmoid function. For a particularly positive or negative value of $x \cdot w + b$, the result will be nearly the same as with the perceptron (i.e., near 0 or 1). For values close to the boundary of the separating hyperplane, values near 0.5 will be emitted.

$$\sigma(x \cdot w + b) = \frac{1}{1 + e^{-(x \cdot w + b)}}$$

- ▶ Many more activation functions available, that you can use based on your need.
 - ▶ hyperbolic tan, rectified linear unit, leaky rectified linear unit, maxout

Neural Network

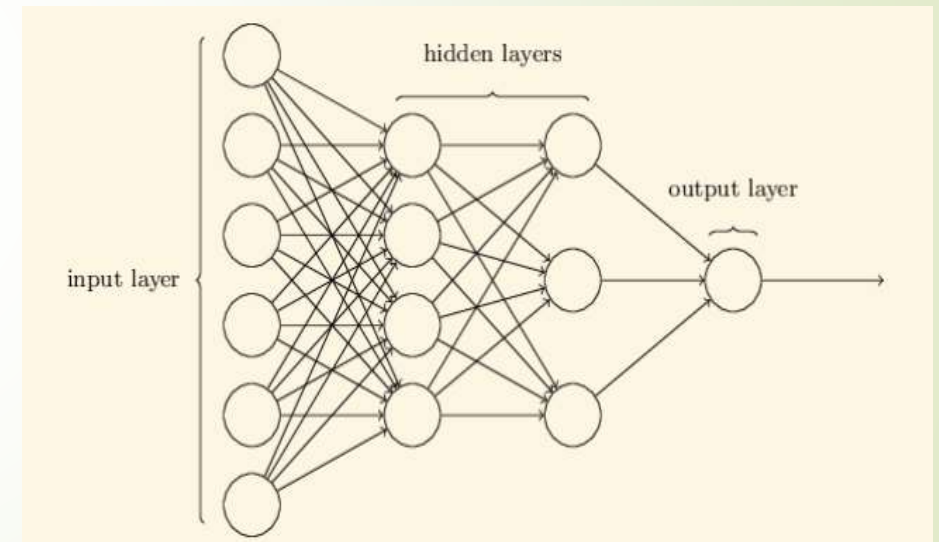
- ▶ Problem with single perceptron is, if you have very high dimensional data, it will not work.
 - ▶ For example you have 784 variables (pixels) from image. Your hyperplane need to be very complex
- ▶ In such cases you have to use networks of perceptron's or neurons

A Network of Perceptron's

What is Happening

- If you see individual neural level:
 - They become active or deactivate based on inputs
- When we see at layer level:
 - A layer is allowing a group of features
 - In this particular example our hidden layers are actually separating features. As a result, 2nd layer sees less feature than input layer, and 3rd layer sees less feature than 2nd layer, and output layer only get features that will help it to make a very simple decision

Neural Network



How they learn

- Cost Function/ Loss Function
 - Basically we train a network
 - During the training process, we give it a input x and let it generate an output $y_{\sim}$.
 - We know the real output y . Using that we calculate the error or loss, and **back-propagate** it to every neuron, use **gradient descent** to adjust there weights so we make less error next time
 - A search algorithm

Functions

Cost Functions

$$C(w, b) = \frac{1}{2n} \sum_i (y_i - \hat{y}(x_i))^2$$

Or, for classification, the hinge loss:

$$C(w, b) = \sum_i [1 - y_i \cdot \hat{y}(x_i)]_+$$

Gradient Descent

$$w_{k+1} = w_k - \eta \cdot \nabla_w C$$

$$b_{k+1} = b_k - \eta \cdot \nabla_b C$$

Question

Can you relate how the search algorithms you learnt so far work here?

Why Neural Networks

- Consider the MNIST digit dataset, you need a very complex model to learn different characters hyperplane.
- Due to complex nature, automatic feature selection capability from loss, Neural Network is very good at this
- Other classifiers can also do these, but you may need to do a lot of feature engineering [In NN hidden layers doing that automatically]

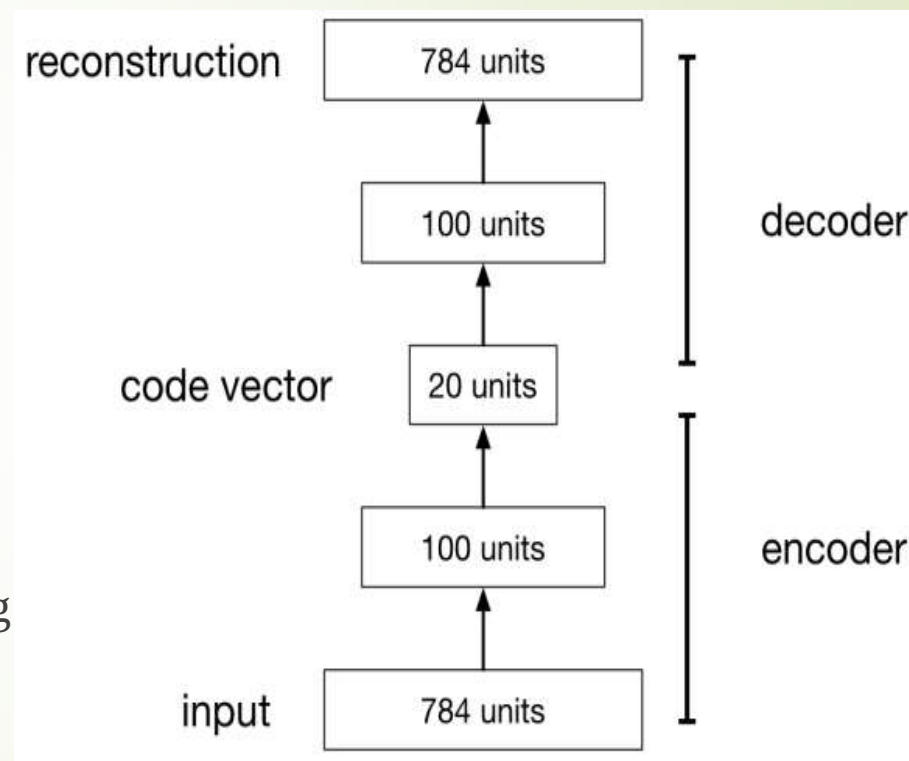


Simplified Representation in 2D

Autoencoders

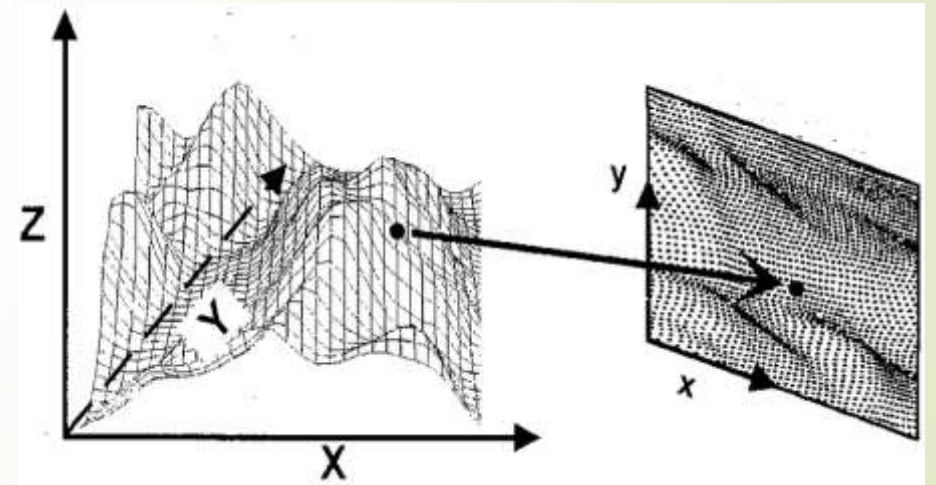
Autoencoders

- An Autoencoder is a feed-forward neural net whose job it is to take an input $\mathbf{x}$ and reconstruct $\mathbf{x}$.
- To make this non-trivial, we need to add a bottleneck layer whose dimension is much smaller than the input.
- Basically, what is happening here, we are reducing 784 dimension input to only 20 dimension.
 - We are compressing features, only trying to keep important ones
 - Next we are trying to reproduce that using only those important ones



Real Life Similarity

- You take a picture (3D- $\rightarrow$ 2D)
- Create a 3D print from the 2D plot



VAEs two main components:

1. Encoder (Inference Model)

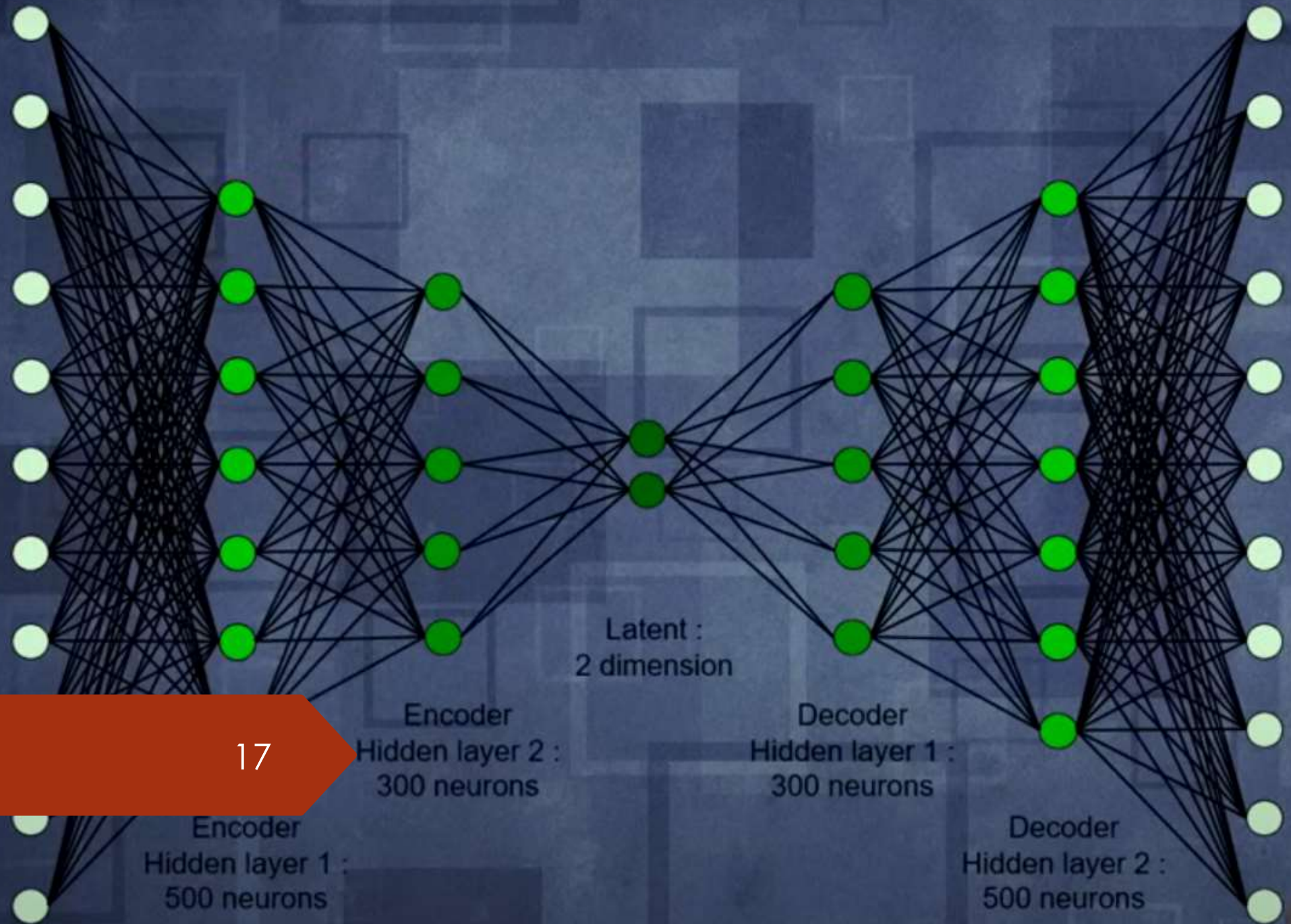
1. Maps input data x to a latent (**hidden**) space z .
2. Instead of outputting a single point, it outputs **parameters of a probability distribution** (typically Gaussian: mean μ and variance σ^2).
3. The latent variable z is sampled from this distribution:

$$z \sim N(\mu, \sigma^2)$$

2. Decoder (Generative Model)

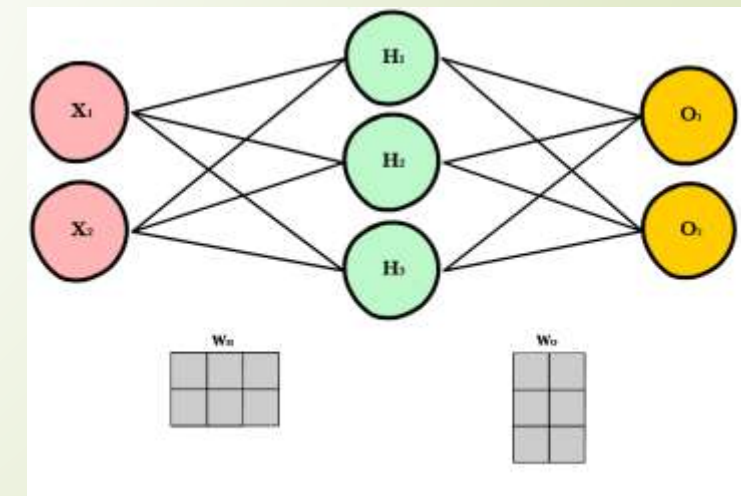
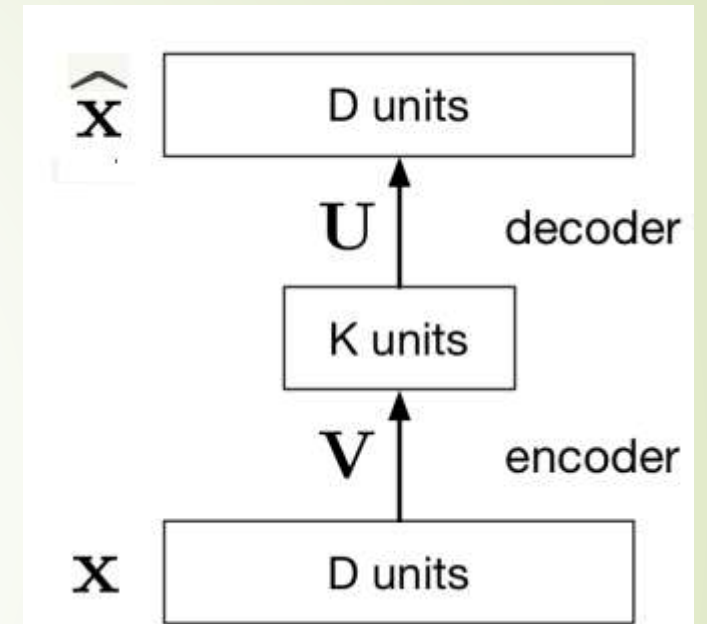
1. Reconstructs the input data from the sampled latent variable z .
2. Learns to generate realistic data by maximizing the likelihood $p(x|z)$.

Reduce
Dimensionality to a
Latent
space then
regenerate



How Autoencoders Works

- $\mathbf{x}$: Input vector or array
- $\hat{\mathbf{x}}$: Output vector or array
- $\mathbf{V}$: A matrix of weights. For example, if D has 3 neurons, K has 2 neuron, for every connection there will be a weight, and it is a $3 * 2$ weight matrix
- $\mathbf{U}$: Another matrix of weights. For example if D has 3 neurons, K has 2 neuron, for every connection there will be a weight and it is a $2 * 3$ weight matrix

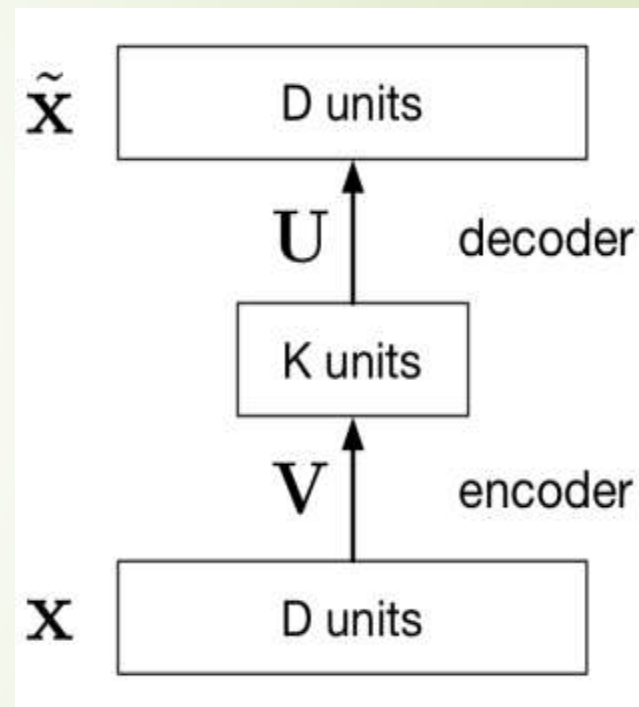


How it works

- **Vx :** A matrix multiplication project D dimensional x to k dimensional plane. Shown for 3 to 2.

Dimensionality Reduction

- **Ux :** Project k dimensional hidden unit to D dimensional $\hat{x}$
- Overall Output is: $\hat{x} = UVx$ [A linear function]
- How it learn:
 - Learn the U, V matrix or all weights so that we get minimum loss
 - $L(x, \hat{x}) = ||x - \hat{x}||^2$
 - We minimize L to learn U, V



Why Autoencoders?

- Map high-dimensional data to two dimensions for visualization
- Compression
- Learn abstract features in an unsupervised way so you can apply them to a supervised task
 - Unlabeled data can be much more plentiful than labeled data
- Learn a semantically meaningful representation where you can, e.g., interpolate between different images.

Some limitations of Autoencoders

Autoencoders (AEs) are powerful tools for unsupervised learning, dimensionality reduction, and feature extraction, but they have several key limitations compared to more advanced models like **Variational Autoencoders (VAEs)** or **Generative Adversarial Networks (GANs)**. Here are some major limitations:

1. Deterministic Latent Space (No Generative Capability)

- **Problem:** Traditional autoencoders learn a **fixed mapping** from input to latent space, meaning they don't model data probabilistically.
- **Consequence:**
 - Cannot generate new data samples by sampling from the latent space (unlike VAEs or GANs).
 - The latent space may have "gaps" or discontinuities, making interpolation unreliable.

Some limitations of Autoencoders

2. Overfitting to Training Data

- **Problem:** If the encoder/decoder is too powerful (e.g., very deep), the model can simply memorize inputs instead of learning useful features.
- **Consequence:**
 - Poor generalization to unseen data.
 - Reconstructions may be near-perfect on training data but fail on test data.

Some limitations of Autoencoders

3. Blurry Reconstructions

- **Problem:** Autoencoders trained with mean squared error (MSE) loss tend to produce **averaged/blurry outputs**.
- **Reason:**
 - MSE penalizes large errors but encourages "safe" predictions (e.g., averaging possible outputs).
 - Unlike GANs, they lack adversarial training to produce sharp, realistic samples.

Some limitations of Autoencoders

4. No Control Over Latent Space Structure

- **Problem:** The latent space isn't explicitly regularized (unlike VAEs, which use KL divergence to enforce a Gaussian prior).
- **Consequence:**
 - Latent representations may be **disorganized** or **uninterpretable**.
 - Similar inputs might map to distant latent points (no smoothness guarantees).

Some limitations of Autoencoders

4. No Control Over Latent Space Structure

- **Problem:** The latent space isn't explicitly regularized (unlike VAEs, which use KL divergence to enforce a Gaussian prior).
- **Consequence:**
 - Latent representations may be **disorganized** or **uninterpretable**.
 - Similar inputs might map to distant latent points (no smoothness guarantees).

Some limitations of Autoencoders

5. Difficulty Handling Noisy or Missing Data

- **Problem:** Standard autoencoders assume clean, complete input data.
- **Consequence:**
 - Performance degrades with noisy or partial inputs (unlike **denoising autoencoders** or VAEs, which are more robust).

Some limitations of Autoencoders

6. Scalability to High-Dimensional Data

- **Problem:** For very high-dimensional data (e.g., high-res images), autoencoders may struggle to learn compact representations.
- **Consequence:**
 - Requires careful architecture tuning (e.g., convolutional layers for images).
 - Bottleneck layer size becomes critical—too small loses information; too large defeats the purpose of compression.

Some limitations of Autoencoders

7. Lack of Probabilistic Interpretation

- **Problem:** Autoencoders don't model uncertainty in the latent space or reconstructions.
- **Consequence:**
 - Cannot estimate likelihoods or confidence scores for generated samples.
 - Hard to use in probabilistic frameworks (e.g., Bayesian deep learning).

When to Use Autoencoders Anyway?

- **Dimensionality reduction** (alternative to PCA).
- **Anomaly detection** (reconstruction error as an outlier score).
- **Pre-training** (e.g., initializing weights for supervised tasks).
- For **generative modeling**, however, **VAEs** or **GANs** are typically preferred.

Comparison with VAEs

Limitation	Autoencoder (AE)	Variational Autoencoder (VAE)
Latent Space	Deterministic	Probabilistic (sampling)
Generative	No	Yes
Regularization	None	KL divergence (Gaussian prior)
Reconstructions	Often blurry	Still blurry, but more structured

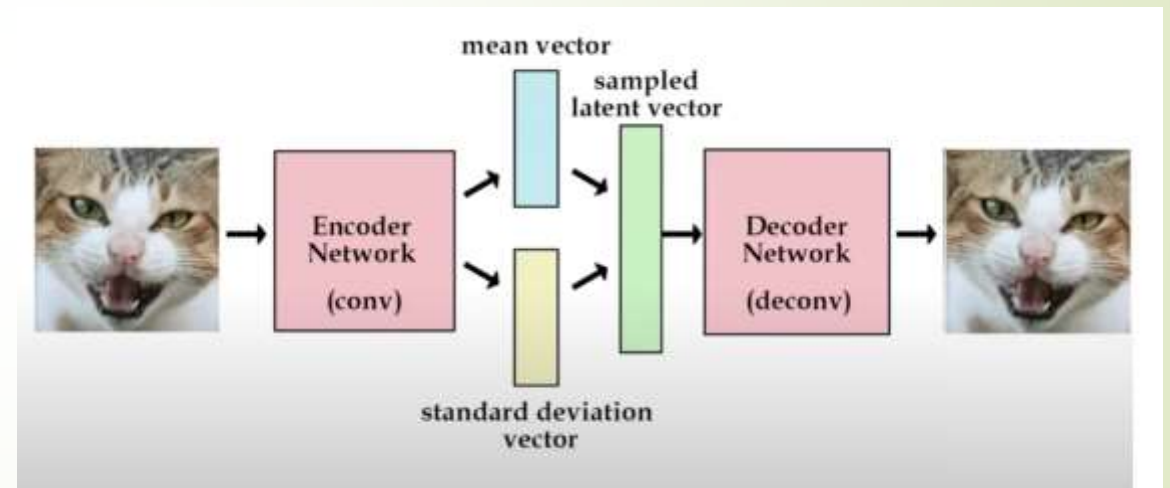
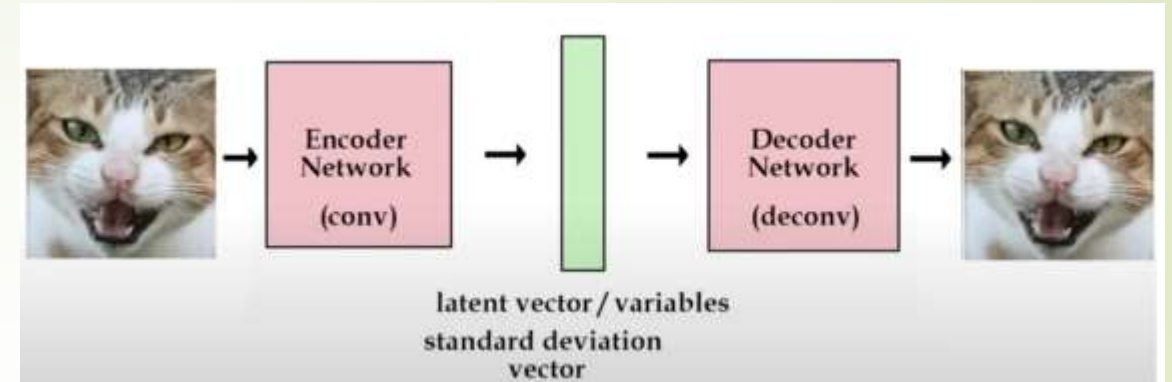
(1) Encoding (Inference)

- Input $\mathbf{x}$ is passed through the encoder.
- The encoder outputs:
 - Mean vector μ
 - Log-variance vector $\log\sigma^2$ (for numerical stability).
- The latent vector $\mathbf{z}$ is sampled using the **reparameterization trick** (see below).

How Variational Auto encoder works

(2) Reparameterization Trick

- Direct sampling $z \sim N(\mu, \sigma^2)$ is non-differentiable.
- Instead, we compute:
 $z = \mu + \sigma \odot \epsilon, \epsilon \sim N(0, I)$ where $\odot$ is element-wise multiplication.
- This allows gradients to flow through μ and σ during backpropagation.



$$\mathcal{L}(\theta, \phi; \mathbf{x}, \mathbf{z}) = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p_{\theta}(\mathbf{x}|\mathbf{z})] - D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z}))$$

Reconstruction

Loss

KL Divergence

Stay close to

Normal(0,1)

33

Mathematical Representation of VAEs

We will try to force the latent distribution **mean** to be relatively **close** to a **mean of zero** and a **standard deviation** of **one** and so finally

(3) Decoding (Generation)

- The sampled $\mathbf{z}$ is passed through the decoder.
- The decoder outputs parameters of $p_{\theta}(\mathbf{x}|\mathbf{z})$ (e.g., mean pixel values for images).
- For binary data (e.g., MNIST), a Bernoulli distribution is often used:

$$p_{\theta}(\mathbf{x}|\mathbf{z}) = \prod_i x_i^{\hat{x}_i} (1 - x_i)^{1-\hat{x}_i}$$

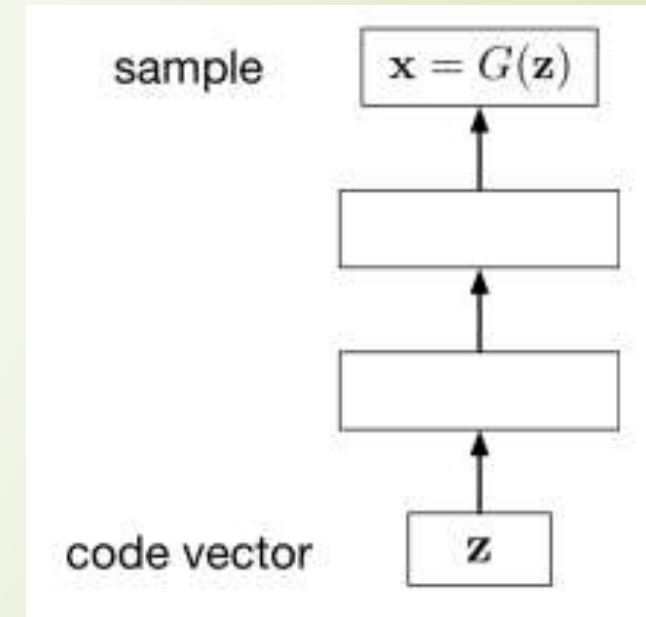
- where $\hat{x}_i$ is the decoder's output probability.

4. Why VAEs Work Better Than Standard Autoencoders

Feature	Standard Autoencoder (AE)	Variational Autoencoder (VAE)
Latent Space	Deterministic (fixed)	Probabilistic (sampling)
Generative?	✗ No	✓ Yes
Regularization	None	KL divergence (Gaussian prior)
Interpolation	Unreliable (gaps)	Smooth transitions
Use Cases	Compression, denoising	Generation, representation learning

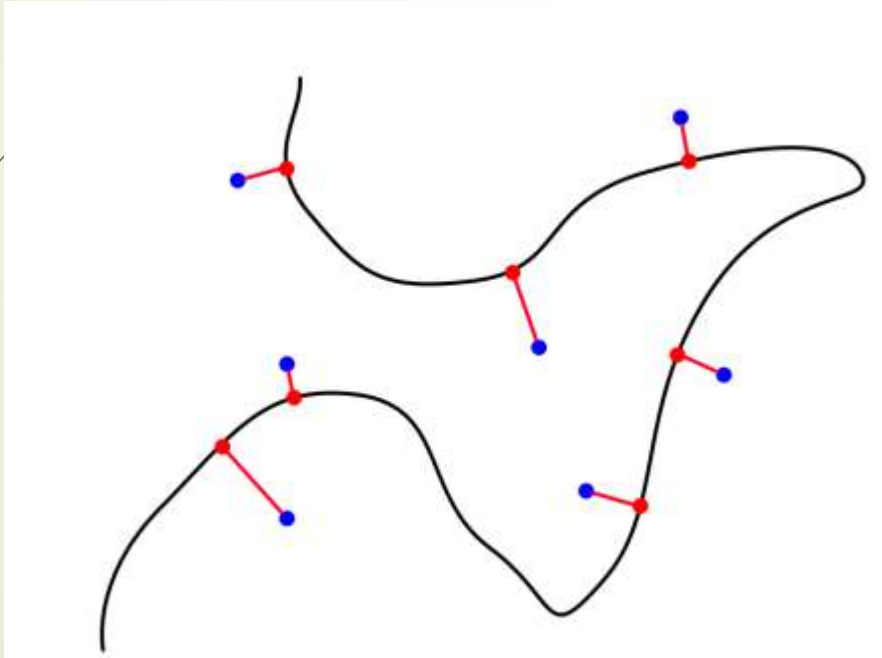
Generative Models

- One of the goals of unsupervised learning is to learn representations of images, sentences, etc.
- A generative model is a function of representation z .
 - $x = G(z)$ [G is the generator function]
- Example: VAE, GAN

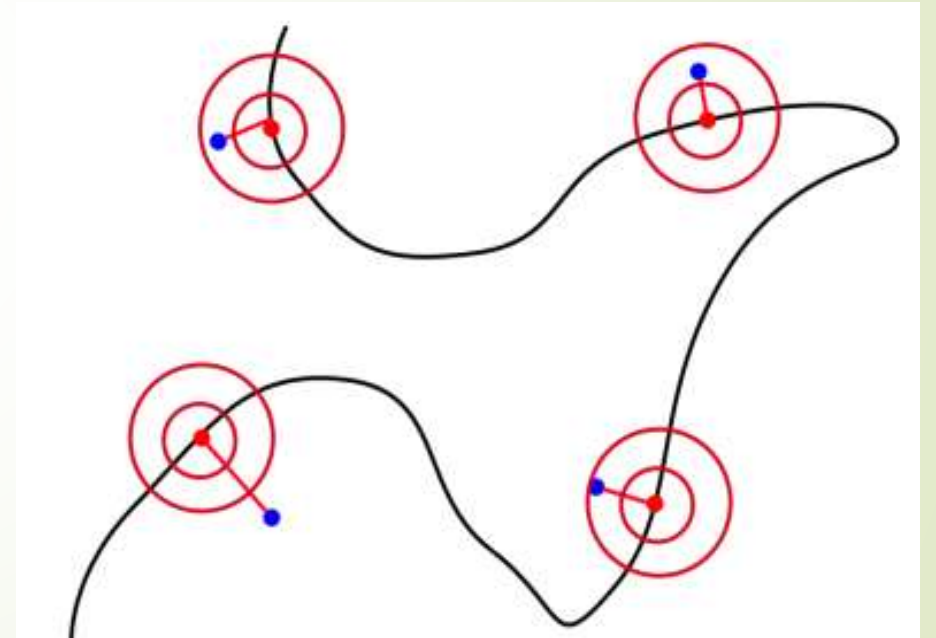


How Learning Distribution Help

Normal AE



VAE



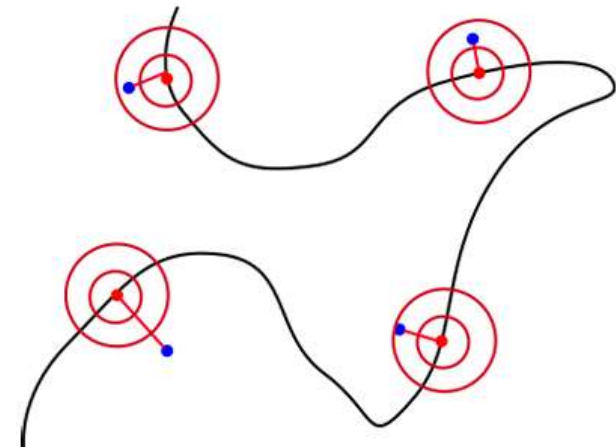
- Consider training a generator network with maximum likelihood.

$$p(\mathbf{x}) = \int p(\mathbf{z})p(\mathbf{x} | \mathbf{z}) d\mathbf{z}$$

- One problem: if $\mathbf{z}$ is low-dimensional and the decoder is deterministic, then $p(\mathbf{x}) = 0$ almost everywhere!
 - The model only generates samples over a low-dimensional sub-manifold of $\mathcal{X}$.
- Solution: define a noisy observation model, e.g.

$$p(\mathbf{x} | \mathbf{z}) = \mathcal{N}(\mathbf{x}; G_{\theta}(\mathbf{z}), \eta \mathbf{I}),$$

where G_{θ} is the function computed by the decoder with parameters θ .

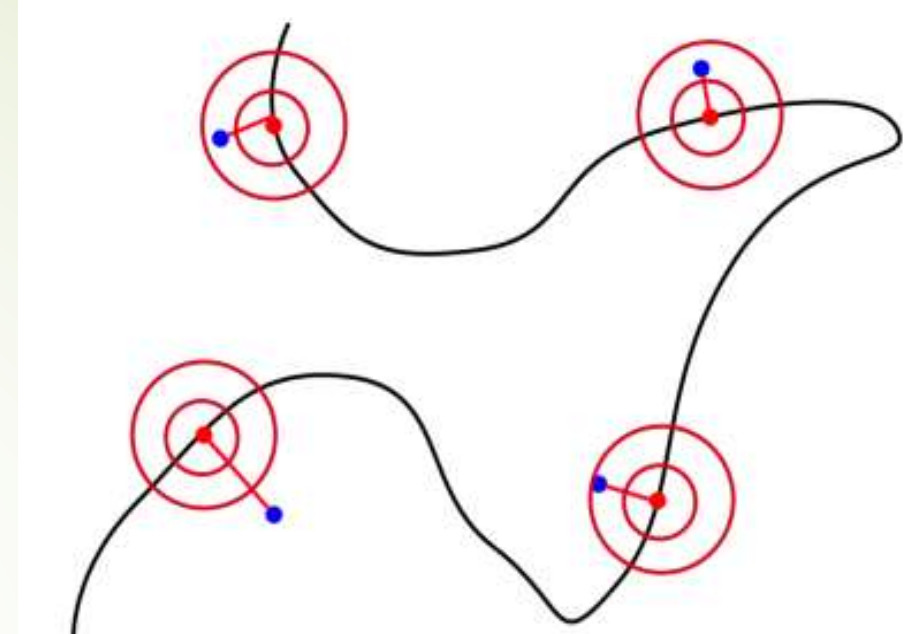


Example

- You reduced 784 pixel character images to 20 dimension using a normal autoencoder and a VAE
- Normal encoder part is giving you a 20 dimensional mean value (or just the cluster center). So when decoder will reproduce, it will be some sample near that mean
- But, in VAE it is giving you a variance also. So, basically now you can generate some sample you did not even see (due to variance), but will also be similar to an image cluster
- Example from 2D cluster at page 12

How VAE Learn Distribution

- VAE based on variational inference
- Decoder does is calculate $p(z | x) = p(x, z) / p(x)$
- What generator part does is:
 - $p(x) = \int p(z)p(x|z)dz$
- A noisy observation model is learnt
 - $p(x|z) = N(x; G_\theta(z), \eta I)$
 - θ is μ and σ learnt by the decoder

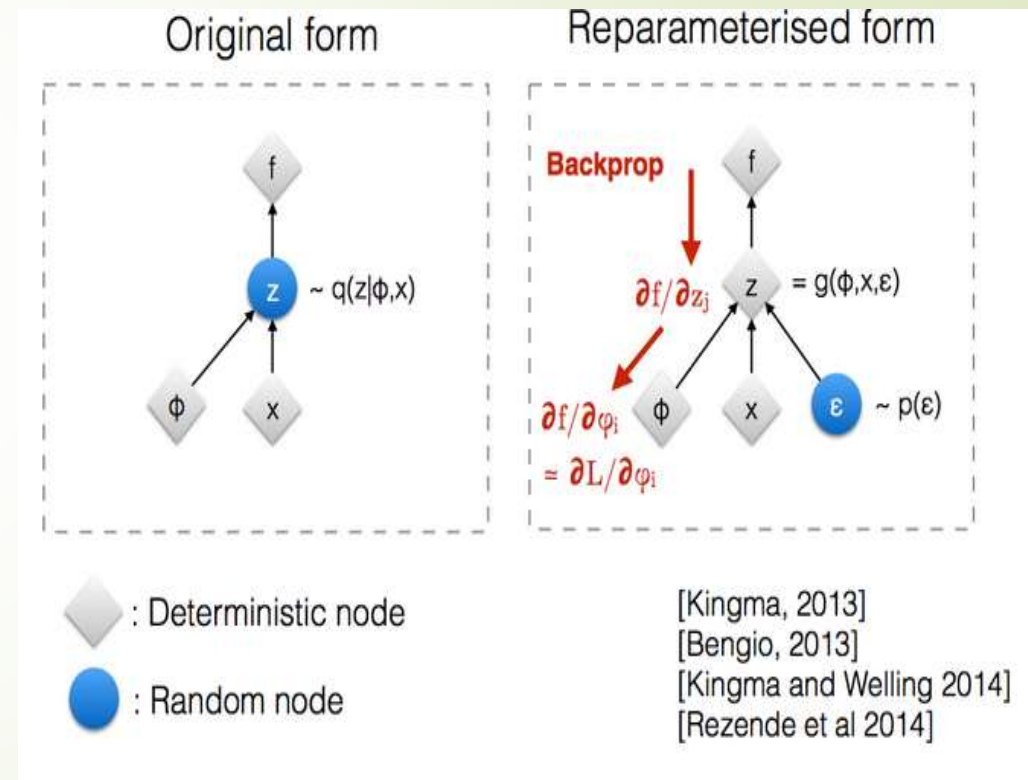


How VAE Learn Distribution(Not Covered)

- Direct computation of $p(x)$ is costly
- It introduce a further function to approximate the posterior distribution as
 - $q_{\phi}(z|x) \approx p_{\theta}(z|x)$
 - Approximate $q(z)$ which is known to $p(z|x)$
- the idea is to jointly optimize the generative model parameters θ to reduce the reconstruction error between the input and the output and ϕ to make $q_{\phi}(z|x)$ as close as $p_{\theta}(z|x)$
 - $\text{Min } D_{KL}(q_{\phi}(\cdot|x)||p_{\theta}(\cdot|x))$
- It uses KL divergence to make the parameters similar and define new loss function using ELBO method that does both task at the same time
 - $L_{\theta,\phi}(x) = \log p_{\theta}(x) - D_{KL}(q_{\phi}(\cdot|x)||p_{\theta}(\cdot|x))$

Reparameterization Trick(Not Covered)

- Back propagation won't work on random node z (if we think it as just distribution)
- So z is made into two deterministic node for mean and std and random part separated as Gaussian error



Class Conditional VAE(Not Covered)

- A class-conditional VAE provides the labels to both the encoder and the decoder.
- Since the latent code $\mathbf{z}$ no longer has to model the image category, it can focus on modeling the stylistic features

